

Resolva os seguintes exercicios:

1. Escreva duas funções para gerar números aleatórios:

randomUntil: fornece um número entre zero e menor que um número máximo dado no parâmetro

randomWithin: fornece um número entre um mínimo e um máximo (intervalo fechado) dados nos parâmetros

Exemplos:

randomUntil(10) → 9

randomWithin(0, 5) → 2

2. Escreva duas funções para gerar números aleatórios, utilizando funções anteriores.

randomEven: fornece um número par até um dado máximo (inclusivé)

randomOdd: fornece um número ímpar até um dado máximo (inclusivé)

3. Escreva uma função para verificar se um número inteiro é um quadrado perfeito (i.e. a sua raiz quadrada é um número inteiro). Para tal, podem ser utilizadas as funções sqrt e floor do módulo Math, para calcular a raiz quadrada e o piso de um número real.

Exemplos:

Math.sqrt(9) → 3.0

Math.floor(3.2) → 3.0

4. Escreva uma classe com três funções relacionadas com divisores:

countDivisors: conta quantos divisores tem um número inteiro;

sumProperDivisors: soma os divisores próprios de um número inteiro (exclui o próprio);

isPrime: verifica se um número é primo.

Exemplos:

countDivisors(8) → 4 (1, 2, 4, 8)

sumProperDivisors(6) → 6 (1 + 2 + 3)

`isPrime(7) → true`

5. Escreva uma classe com duas funções relacionadas com números primos:

`countPrimes`: conta quantos números primos existem até um determinado número natural (inclusivé);

`existsPrimeBetween`: verifica se existe algum primo entre dois números naturais (intervalo aberto).

Exemplos:

`countPrimes(7) → 4`

`existsPrimeBetween(5, 9) → true`

6. Escreva uma classe com duas funções relacionadas com números perfeitos. Um número perfeito é aquele cuja soma dos divisores próprios é igual ao próprio número. Por exemplo, 6 é perfeito porque $(1 + 2 + 3)$ é igual a 6.

`isPerfect`: verifica se um número é perfeito;

`countPerfectNumbers`: conta quantos números perfeitos existem até um limite máximo.

Exemplos:

`isPerfect(6) → true`

`countPerfectNumbers(30) → 2 (6, 28)`

7. Defina uma função que recebe como argumento um natural max e devolve a maior diferença entre dois números primos consecutivos até max.

Exemplos:

`largestPrimeDiff(13) → 4`